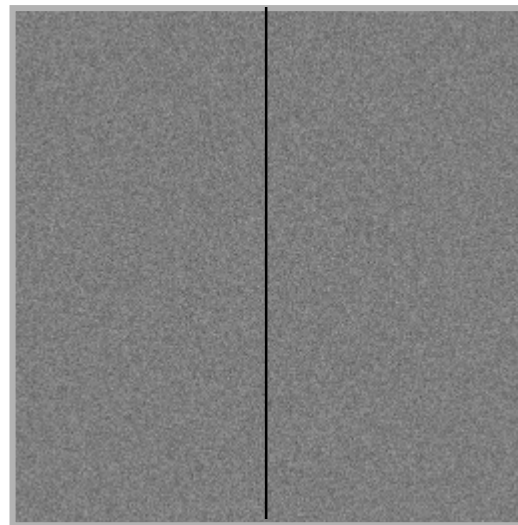


- What is noise?
- How is noise reduction performed?
 - Noise reduction from first principles
 - Neighbourhood operators
 - linear filters (low pass)
 - non-linear filters (median)



image

+



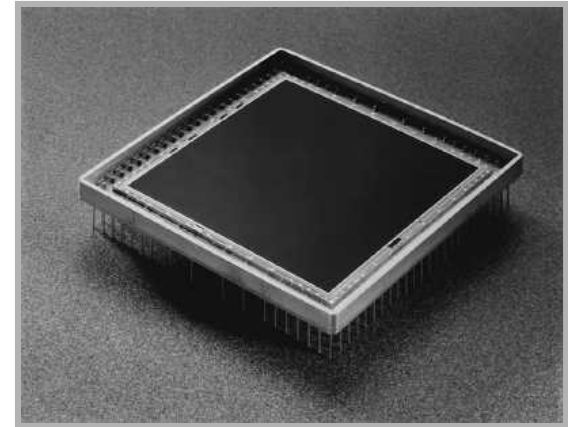
noise

=



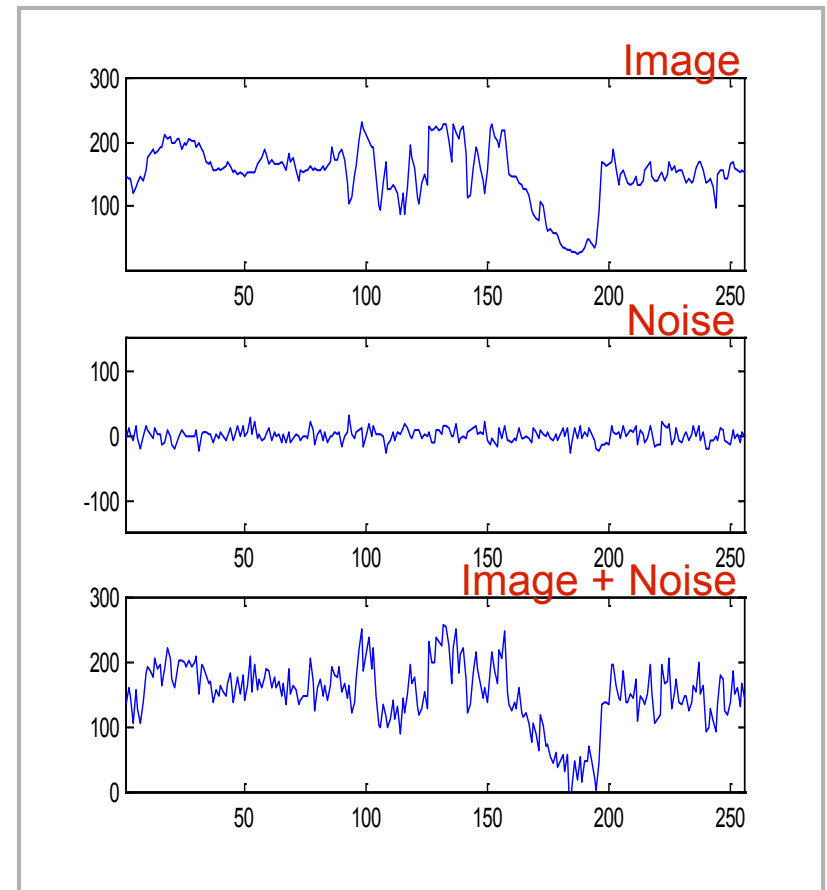
'grainy' image

- Sources of noise = CCD chip.
- Electronic signal fluctuations in detector.
 - Caused by thermal energy.
 - Worse for infra-red sensors.
- Other electronics
- Transmission (analog)

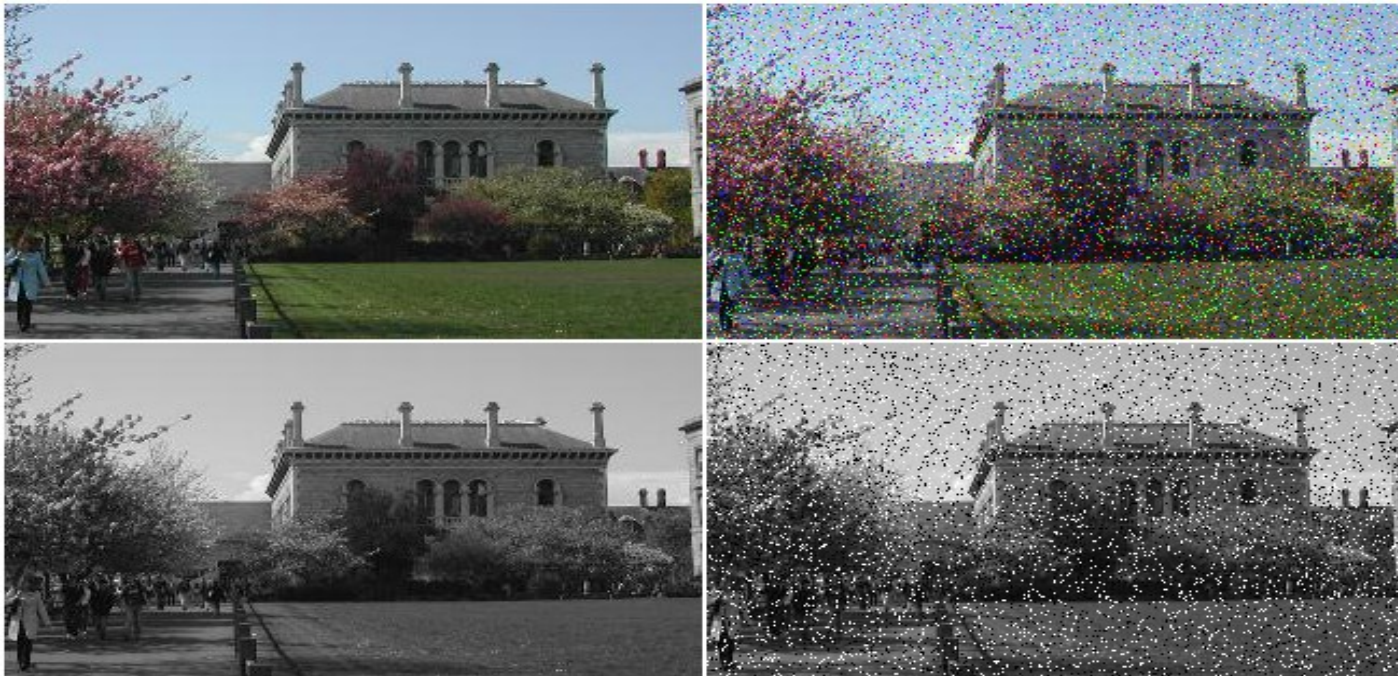


Radiation from the long wavelength IR band is used in most infrared imaging applications

- Plot of image brightness.
 - Noise models:
 - additive
 - $f(i,j) = g(i,j) + v(i,j)$
 - Gaussian
 - salt and pepper
 - multiplicative
 - $f(i,j) = g(i,j) + g(i,j).v(i,j)$
- where
- $f(i,j)$ - acquired signal (noisy)
 - $g(i,j)$ - uncorrupted signal
 - $v(i,j)$ - noise component
- Noise fluctuations are rapid
 - high frequency.



- Impulse noise
 - Noise is maximum or minimum values

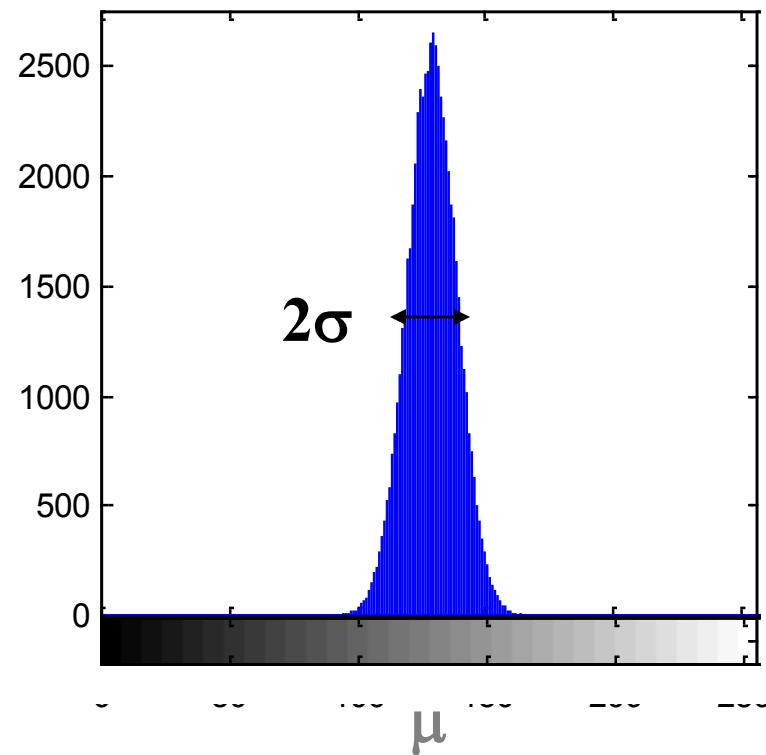


- Good approximation to real noise
- Distribution is Gaussian (mean & standard deviation)



- Plot noise histogram
- Typical noise distribution is normal or Gaussian
- Mean(noise) $\mu = 0$
- Standard deviation σ

$$\eta(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2} \frac{(x-\mu)^2}{\sigma^2}\right]$$



- Noise varies above and below uncorrupted image.

Image

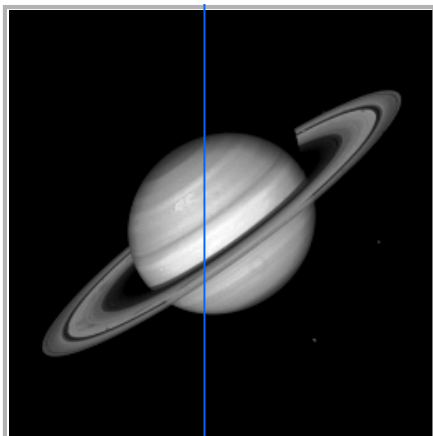
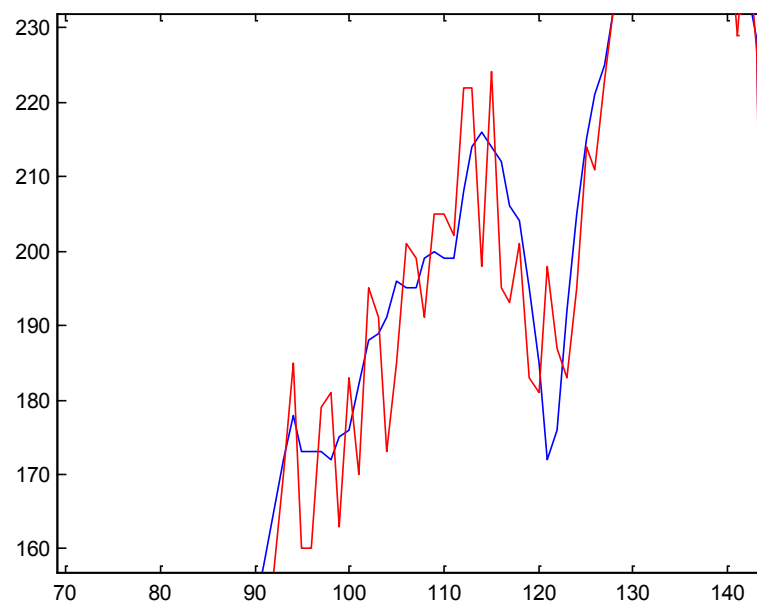
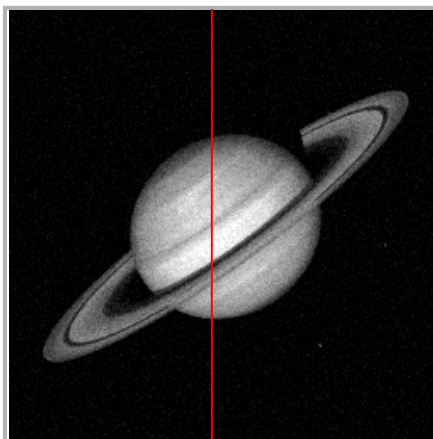


Image
+
Noise



- Removing (?) or reducing noise...
- Linear smoothing transformations
 - frame averaging (=> acquire several frames of static scene)
 - local averaging
 - Gaussian smoothing
- Non-linear transformations
 - median filter
 - rotating mask

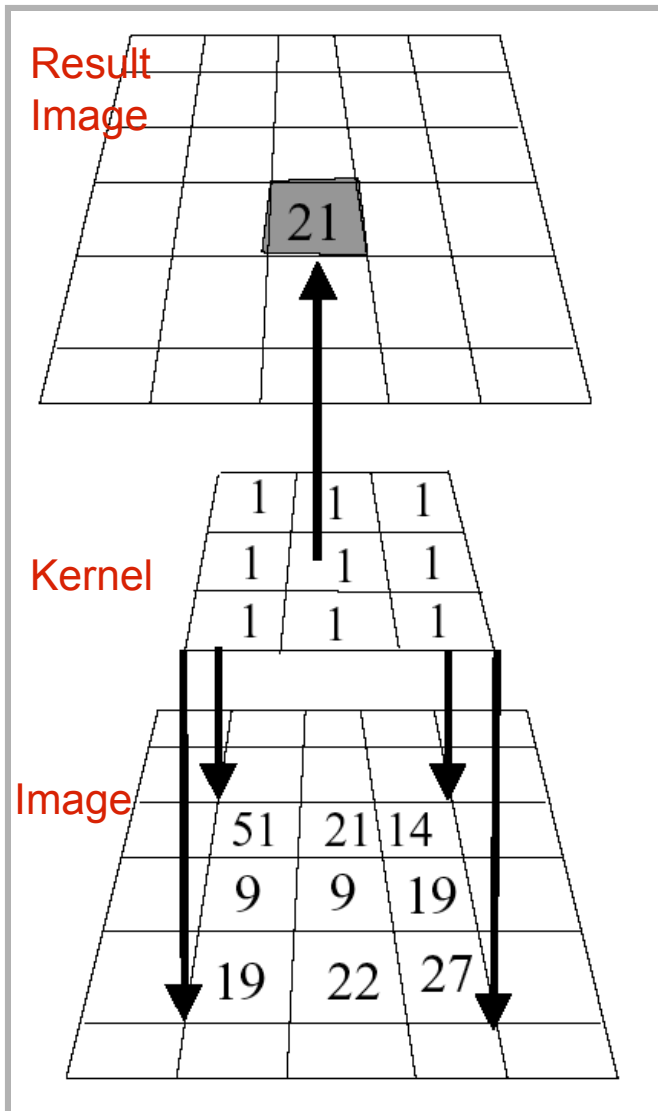
Image Data:

10	12	40	16	19	10
14	22	52	10	55	41
10	14	51	21	14	10
32	22	9	9	19	14
41	18	9	22	27	11
10	7	8	8	4	5

Mask / Filter / Kernel:

1	1	1	1
0	1	1	1
-1	1	1	1
	-1	0	1

$$F \circ I(x, y) = \sum_{j=-N}^N \sum_{i=-N}^N \overset{\text{Filter}}{F(i, j)} \overset{\text{Image}}{I(x+i, y+j)}$$



- Kernel is aligned with pixel in image, multiplicative sum is computed, normalized,
- and stored in result image.
- Process is repeated across image.
- What happens when kernel is near edge of input image?

$$\left[\begin{array}{l} 1*51 + 1*21 + 1*14 + \\ 1*8 + 1*9 + 1*19 + \\ 1*19 + 1*22 + 1*27 \end{array} \right] \left(\frac{1}{9} \right) = 21$$

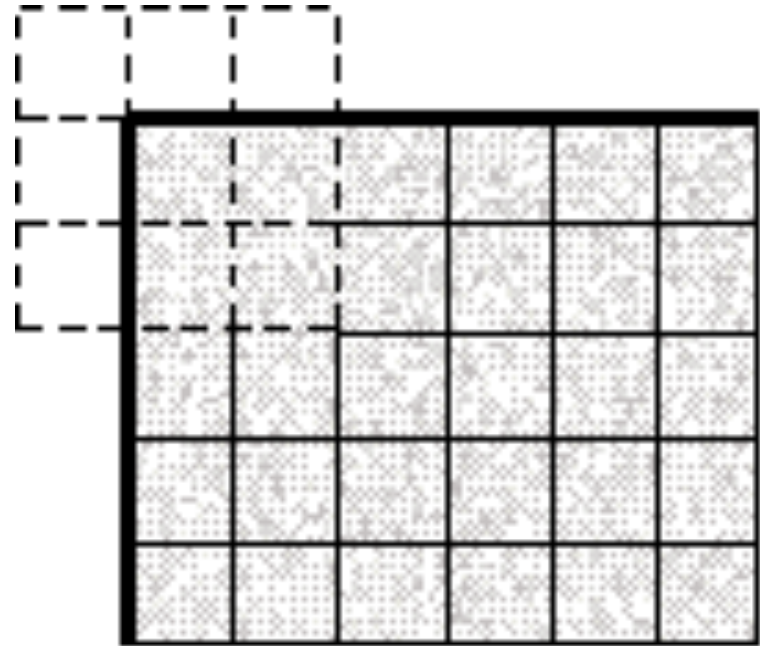
1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

F

← note the factor

- missing samples are zero
- missing samples are gray
- copying last lines
- reflected indexing (mirror)
- circular indexing (periodic)
- reduce size of resulting image

- OpenCV
allows some control on how to do this;
see `CopyMakeBorder()`: copies the source
2D array into the interior of the destination
array and makes a border of the specified
type around the copied area





1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

 F

8	3	4	5
7	6	4	5
4	5	7	8
6	5	5	6

 I

8	8	3	4	5	5
8	8	3	4	5	5
7	7	6	4	5	5
4	4	5	7	8	8
6	6	5	5	6	6
6	6	5	5	6	6

 I with padded boundaries

6.44	5.22	4.33	4.67
5.78	5.33	5.22	5.67
5.56	5.44	5.67	6.00
5.22	5.33	5.78	6.33

 $J = F \circ I$

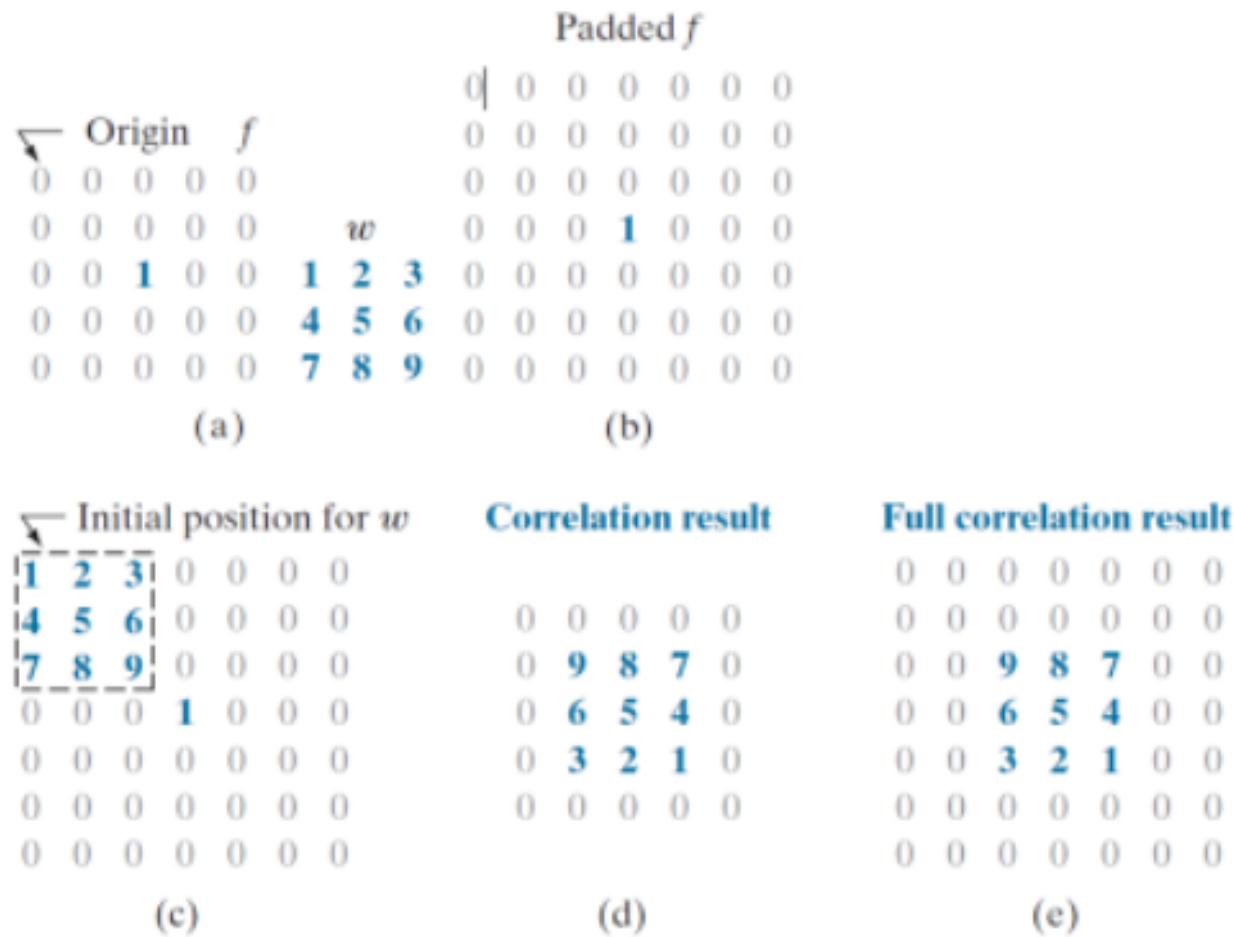
- A **convolution** operation is a correlation where the filter is flipped both horizontally and vertically before being applied to the image:

$$F * I(x, y) = \sum_{j=-N}^N \sum_{i=-N}^N F(i, j) I(x - i, y - j)$$

- Suppose F is a Gaussian or mean kernel.
How does convolution differ from cross-correlation?

- Extremely important concept in computer vision, image processing, signal processing, etc.
- Lots of related mathematics (**we won't do**)
- General idea: reduce a filtering operation to the repeated application of a mask (or filter kernel) to the image
 - Kernel can be thought of as an $N \times N$ image
 - N is usually odd so kernel has a central pixel
- In practice
 - (flip kernel)
 - Align kernel center pixel with an image pixel
 - Pointwise multiply each kernel pixel value with corresponding image pixel value and add results
 - Resulting sum is normalized by kernel weight
 - Result is the value of the pixel centered on the kernel

Correlation



Correlation result

0

0

0

0

0

0

9

8

7

0

0

6

5

4

0

0

3

2

1

0

0

0

0

0

0

(d)

Full correlation result

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

0

9

8

7

0

0

0

0

6

5

4

0

0

0

0

3

2

1

0

0

0

0

0

0

0

0

0

0

0

0

0

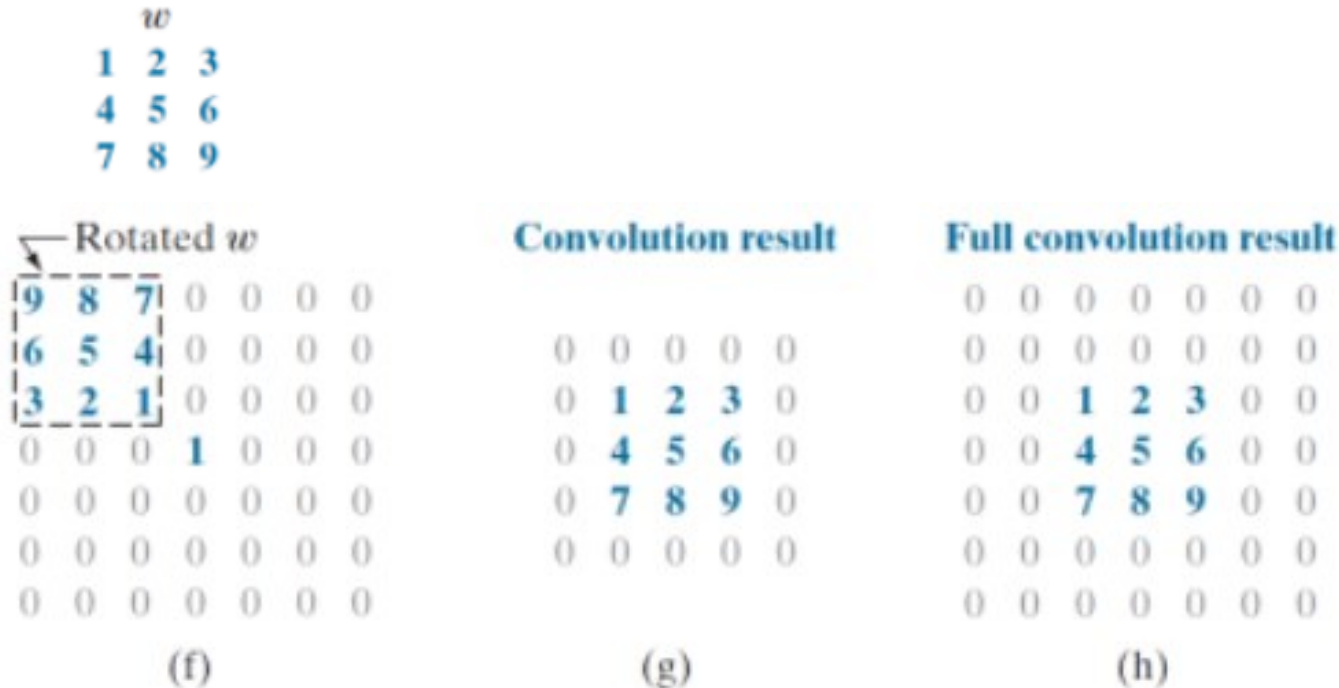
0

0

0

(e)

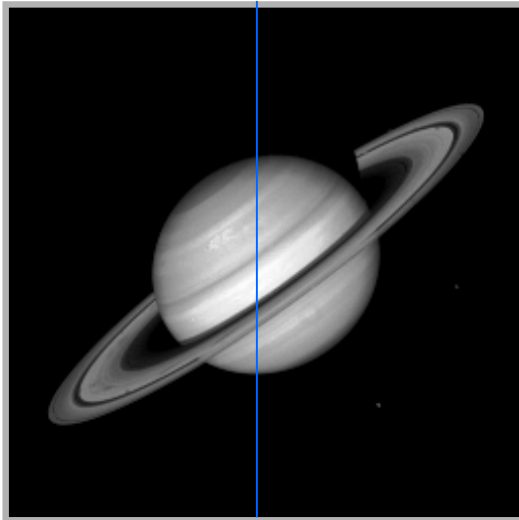
Convolution



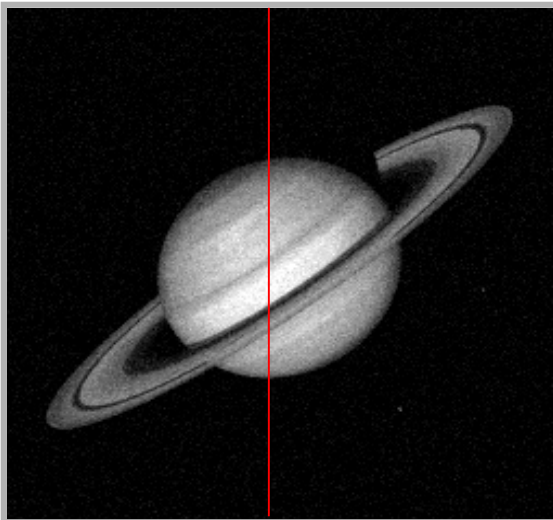
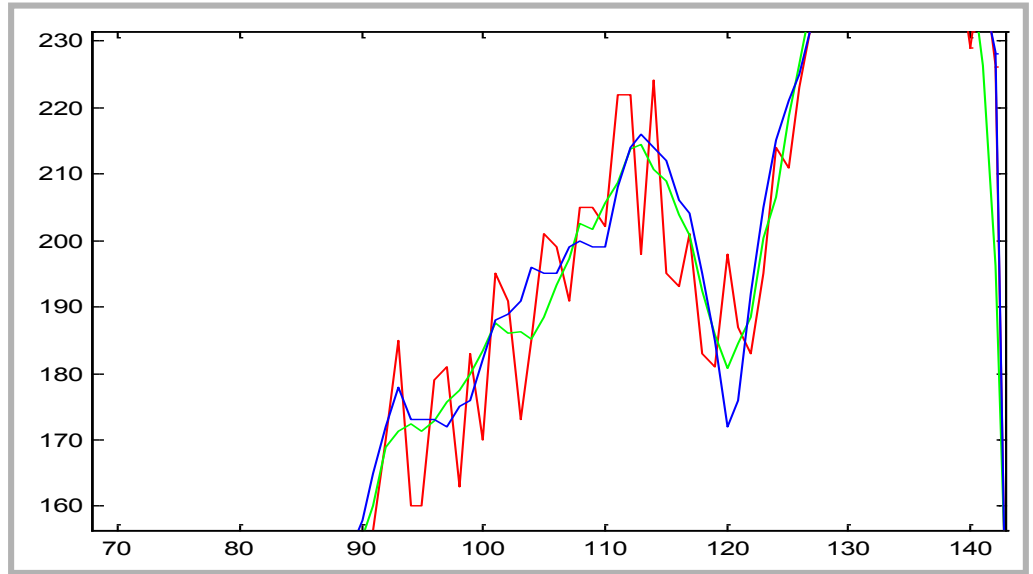
source: <https://towardsdatascience.com/convolution-vs-correlation-af868b6b4fb5>

Notes:

- 1) Correlation and convolution are identical when the filter is symmetric.
- 2) The key difference between the two is that convolution is associative, that is, if F and G are filters, then $F*(G*Img) = (F*G)*Img$
- 3) In general, people use **convolution** for image processing operations such as smoothing, and they use **correlation** to match a template to an image (*see later*).



Uncorrupted Image



Uncorrupted Image + Noise

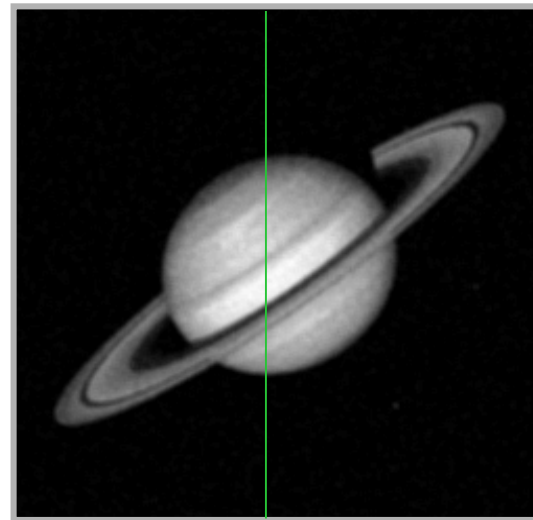


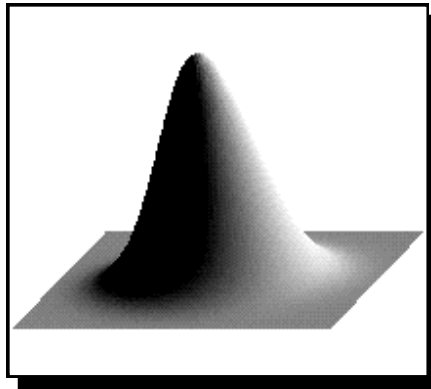
Image + Noise - Blurred

- Technique relies on high frequency noise fluctuations being 'blocked' by filter.
Hence, **low-pass** filter.
- Fine detail in image may also be smoothed.
- Balance between
keeping image fine detail and reducing noise.

- Saturn image
(*previous slide*)
has coarse detail
- Boat image
contains fine detail
- Noise reduced but
fine detail also smoothed



- Smoothing operator should be
 - 'tunable' in what it leaves behind
 - smooth and localized in image space.
- One operator which satisfies these two constraints is the Gaussian

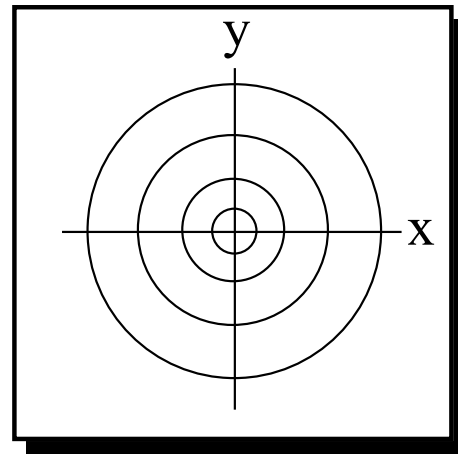


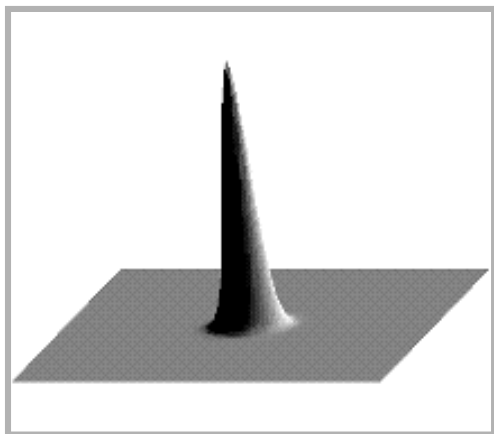
- *OpenCV: Smooth()*

- The two-dimensional Gaussian distribution is defined by:

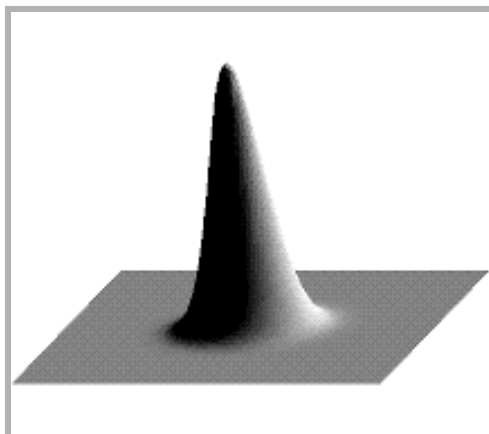
$$G(x,y) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\left[\frac{(x^2 + y^2)}{2 \sigma^2} \right]}$$

- From this distribution, can generate smoothing masks whose width depends upon σ :

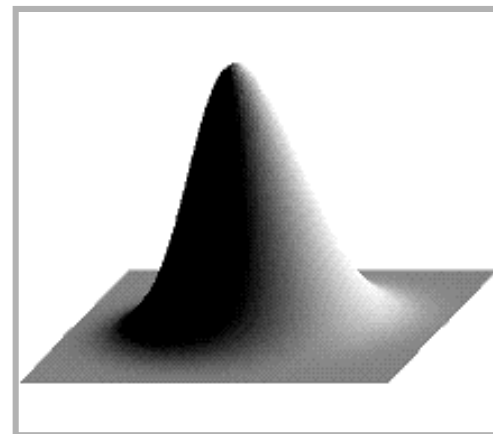




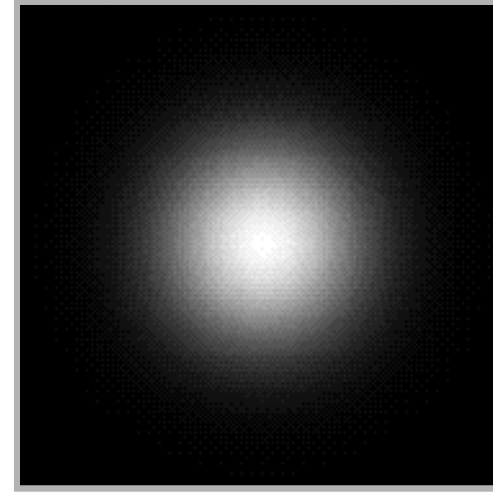
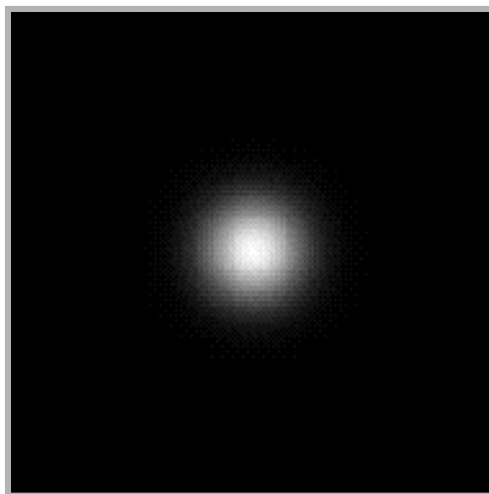
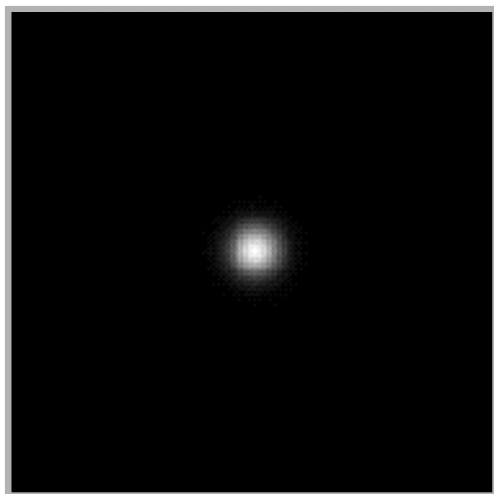
$$\sigma^2 = .25$$



$$\sigma^2 = 1.0$$



$$\sigma^2 = 4.0$$



- Choose $\sigma^2 = 2$ and $n = 7$, then:

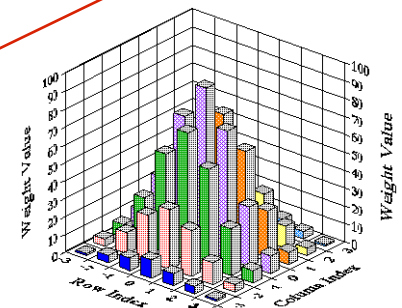
	j						
	-3	-2	-1	0	1	2	3
-3	.011	.039	.082	.105	.082	.039	.011
-2	.039	.135	.287	.368	.287	.135	.039
-1	.082	.287	.606	.779	.606	.287	.082
0	.105	.039	.779	1.000	.779	.368	.105
1	.082	.287	.606	.779	.606	.287	.082
2	.039	.135	.287	.368	.287	.135	.039
3	.011	.039	.082	.105	.082	.039	.011

1	4	7	10	7	4	1
4	12	26	33	26	12	4
7	26	55	71	55	26	7
10	33	71	91	71	33	10
7	26	55	71	55	26	7
4	12	26	33	26	12	4
1	4	7	10	7	4	1

7x7 Gaussian filter

$$\frac{W(1,2)}{k} = \exp\left(-\frac{1^2 + 2^2}{2 \cdot 2}\right)$$

To make this value 1,
choose $k=91$





7x7 Gaussian kernel



15x15 Gaussian kernel

- Gaussian is not the only choice, but it has a number of important properties
 - If we convolve a Gaussian with another Gaussian, the result is a Gaussian
 - This is called linear scale space

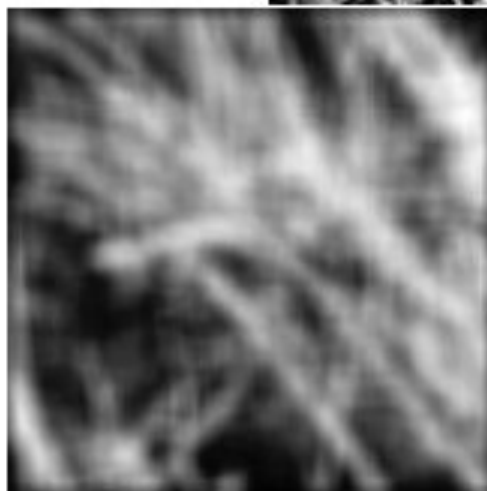
$$G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2 + \sigma_2^2}}.$$

- Efficiency: separable

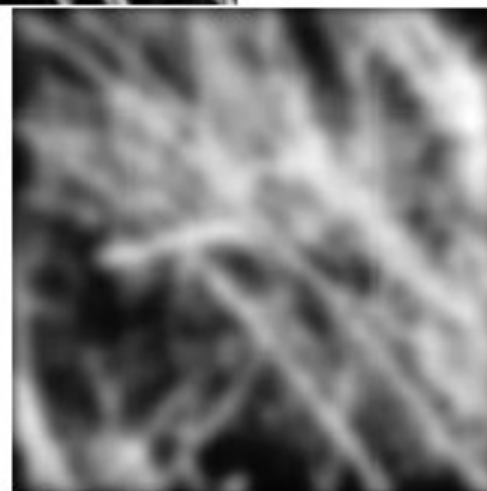
$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(x^2 + y^2)}{2\sigma^2}\right) \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x^2)}{2\sigma^2}\right)\right) \times \left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^2)}{2\sigma^2}\right)\right), \end{aligned}$$



Original image



After mean filtering



After Gaussian filtering

Image

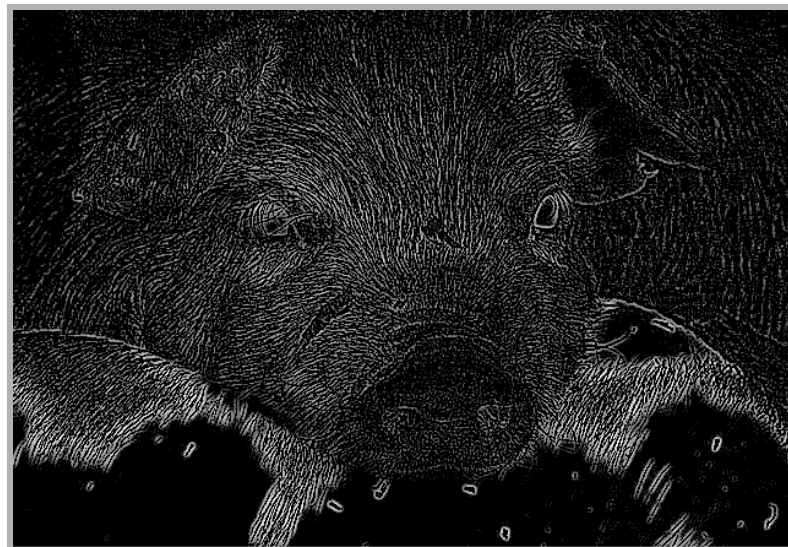


Blurred image



-

=



- Compute median of points in neighborhood
- Nonlinear filter
 - example of a larger class of filters named "rank-filters" (ex: min, median, max)
- Has a tendency to blur detail less than averaging
- Works very well for 'shot' or 'salt and pepper' noise

Linear vs Non-linear

$$\text{Mean}(I1+I2) = \text{Mean}(I1) + \text{Mean}(I2)$$

$$\text{Median}(I1+I2) \neq \text{Median}(I1) + \text{Median}(I2)$$



Original



Low-pass



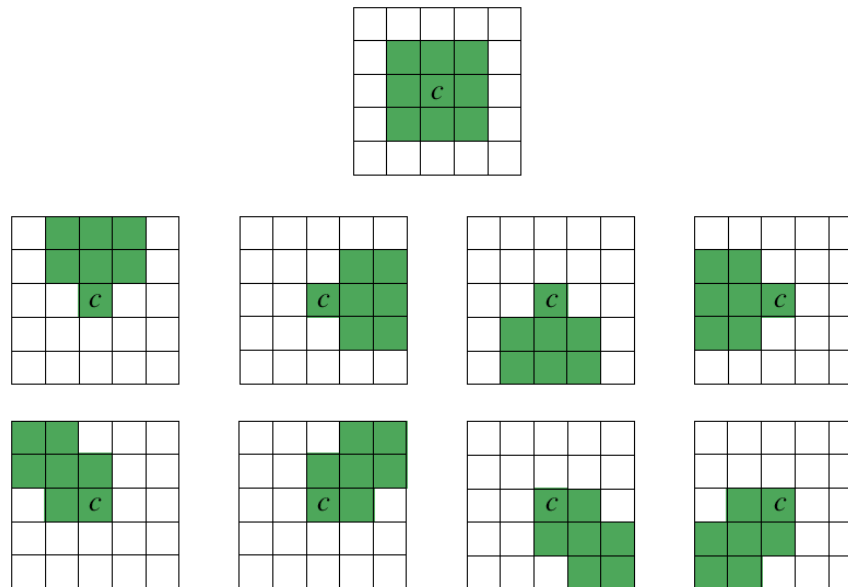
Median

- Mean
 - Linear
 - Signal frequencies shared with noise are lost, resulting in blurring.
 - Impulsive noise is diffused but not removed.
 - It spreads the noise, resulting in blurring.
 - Blurs edges.
- Median
 - Non-linear
 - Does not spread the noise.
 - Can remove spike noise.
 - Preserves (some) edges.
 - Small details (small regions, thin edges, ...) may be lost.
 - Expensive to run.

Smooth (average, blur) an image without disturbing

- sharpness or
- position
of the edges
- Nagao-Matsuyama filter
- Kuwahara filter
- Anisotropic diffusion filters (Perona & Malik,
- Bilateral filtering
- ...

- Calculate the variance within nine subwindows of a 5x5 moving window
- Output value is the **mean** of the **subwindow with the smallest variance**
- Nine subwindows used:



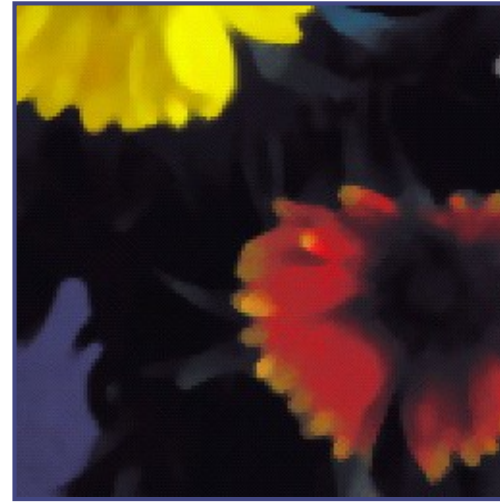
- Principle:
 - divide filter mask into four regions (a, b, c, d).
 - in each compute the mean brightness and the variance
 - the output value of the center pixel (abcd) in the window is the **mean** value of that **region** that has the **smallest variance**.

a	a	ab	b	b
a	a	ab	b	b
ac	ac	abcd	bd	bd
c	c	cd	d	d
c	c	cd	d	d





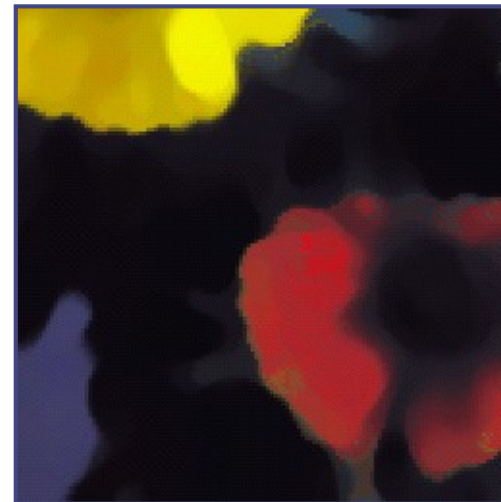
Original



Median (1 iteration)



Kuwahara



Median (10 iterations)

- The anisotropic diffusion is a nonlinear filter that smoothes the intraregions of an image without blurring the strong edges
 - Encourages the **smoothing in homogeneous regions in preference to** smoothing across the boundaries.
- Based on the solution of a partial differential equation, inspired in heat diffusion equation
 - *The algorithm results from the discretization of the non-linear diffusion equation.*
 - *The derivatives are approximated by differences.*
- Diffusion methods average over extended regions by solving partial differential equations, and are therefore inherently iterative. Iteration may raise issues of stability and, depending on the computational architecture, efficiency.
- This algorithm is also used to detect the edges



Original



Perona, Malik, 1987

$$\frac{\partial u}{\partial t} = \operatorname{div} \left(\frac{Du}{|Du|^2 + \lambda^2} \right)$$

Rudin, Osher,
Fatemi, 1992

$$\frac{\partial u}{\partial t} = \operatorname{div} \left(\frac{Du}{|Du|} \right)$$



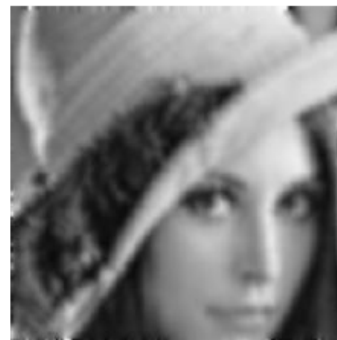
Alvarez, Lions, 1992

$$\frac{\partial u}{\partial t} = \frac{|Du|}{|k * Du|} \operatorname{div} \left(\frac{Du}{|Du|} \right)$$



Weickert, 1994

$$\begin{aligned} \frac{\partial u}{\partial t} &= D^2 u(d, d) \\ d &= SEigen(k * (Du \otimes Du)) \end{aligned}$$



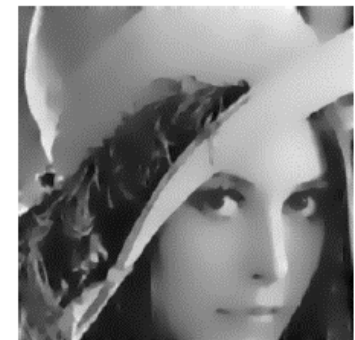
Caselles, Sbert, 1997

$$\frac{\partial u}{\partial t} = \frac{1}{|Du|^2} D^2 u(Du, Du)$$



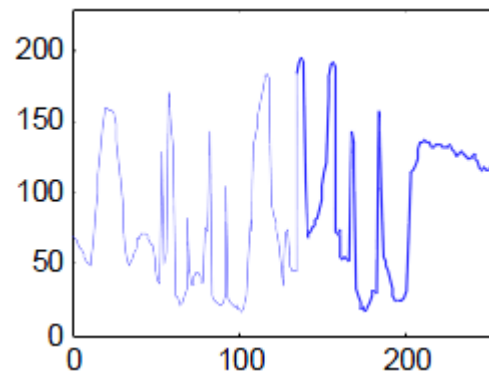
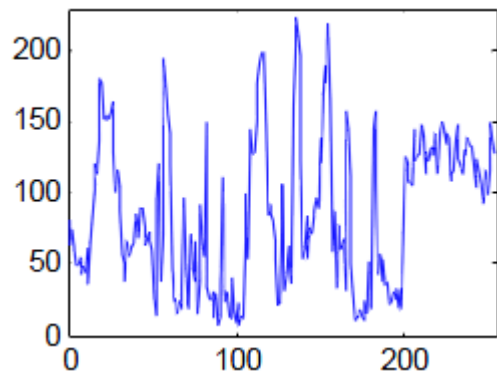
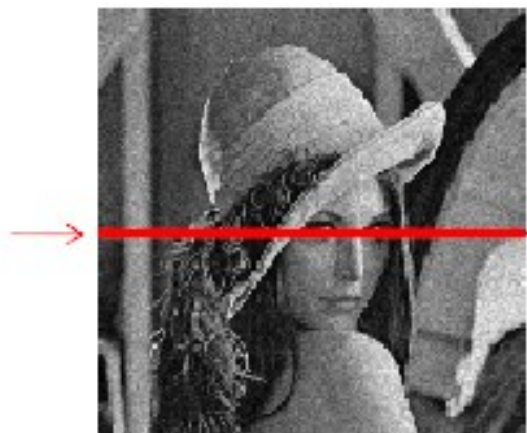
Zhong Carmona, 1998

$$\begin{aligned} \frac{\partial u}{\partial t} &= D^2 u(d, d) \\ d &= SEigen(D^2 u) \end{aligned}$$

Sochen, Kimmel,
Malladi, 1998

$$\frac{\partial u}{\partial t} = \operatorname{div} \left(\frac{Du}{\sqrt{|Du|^2 + 1}} \right)$$

Note: Original Perona-Malik diffusion process is NOT anisotropic, although they erroneously said it was.

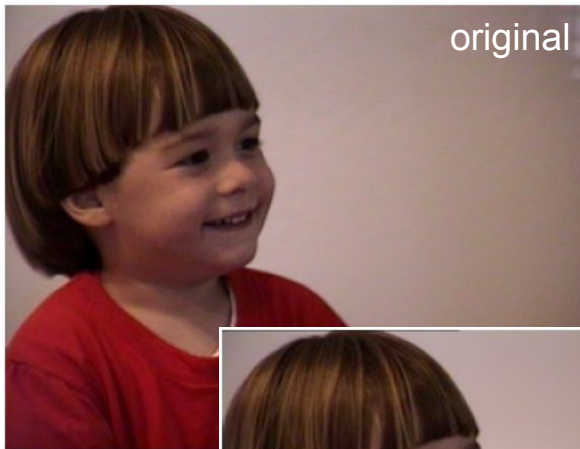


- A bilateral filter is an edge-preserving and noise reducing smoothing filter.
- Traditional filtering is **domain filtering**, and enforces closeness by weighing pixel values with weights that fall off with distance.
- Similarly, we define **range filtering**, which averages image values with weights that decay with dissimilarity.
- **Range filters**
 - are nonlinear because their weights depend on image intensity or color
 - preserve edges
 - by themselves, may distort an image's color map
(see C. Tomasi et al., Bilateral Filtering for Gray and Color, ICCV 1998)
- **Bilateral filtering** combines range and domain filtering.
- The intensity value at each pixel in an image is replaced by a **weighted average of intensity values from nearby pixels**.
- This weight is based on a Gaussian distribution.
The **weights** depend not only on Euclidean distance but also on the radiometric differences (e.g. color intensity).
- It preserves sharp edges by systematically looping through each pixel and attributing weights to the adjacent pixels accordingly.
- *OpenCV: Smooth()*

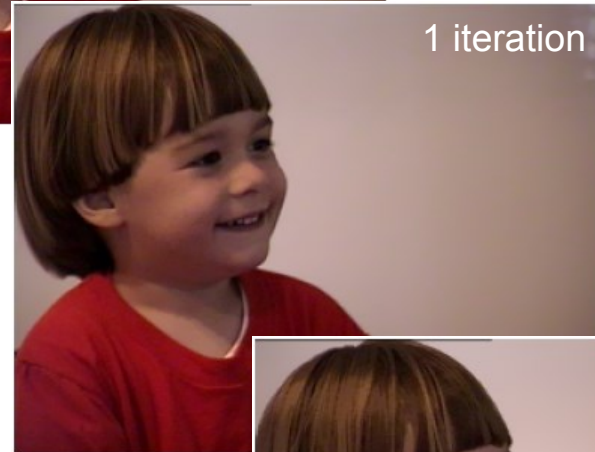
original



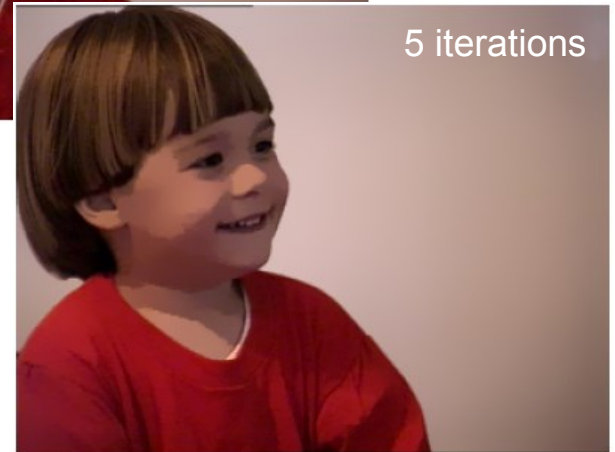
original



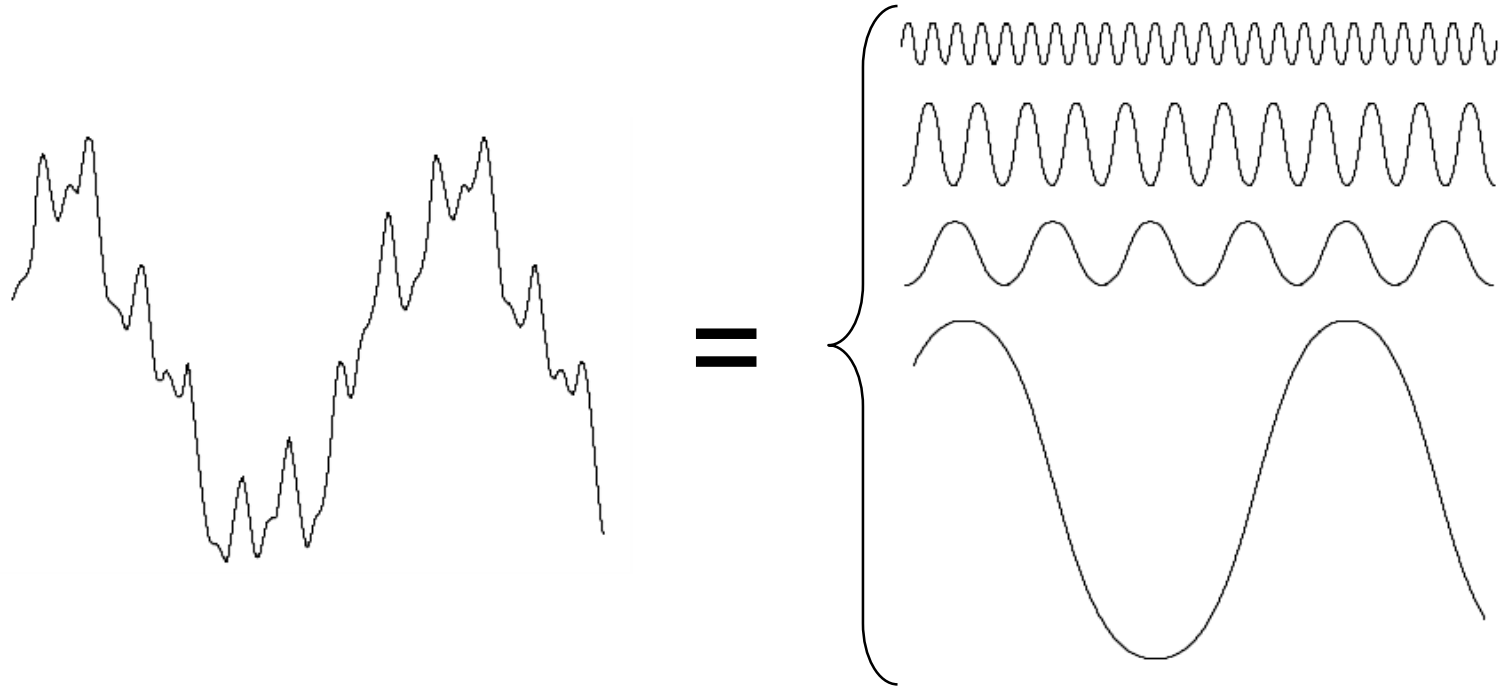
1 iteration



5 iterations



- An image can be represented in the frequency domain, using the Fourier Transform (FT).
- The FT encodes the amplitude and phase of each frequency component.
- The values near the origin of the transformed space are called low-frequency components of the FT, and those distant from the origin are the high-frequency components.
- Convolution in the image domain corresponds to multiplication in the spatial frequency domain.
- Therefore, convolution with large filters, which would normally be expensive in the image domain, can be implemented efficiently using the Fast Fourier Transform (FFT).
- This is an important technique in many image processing applications.



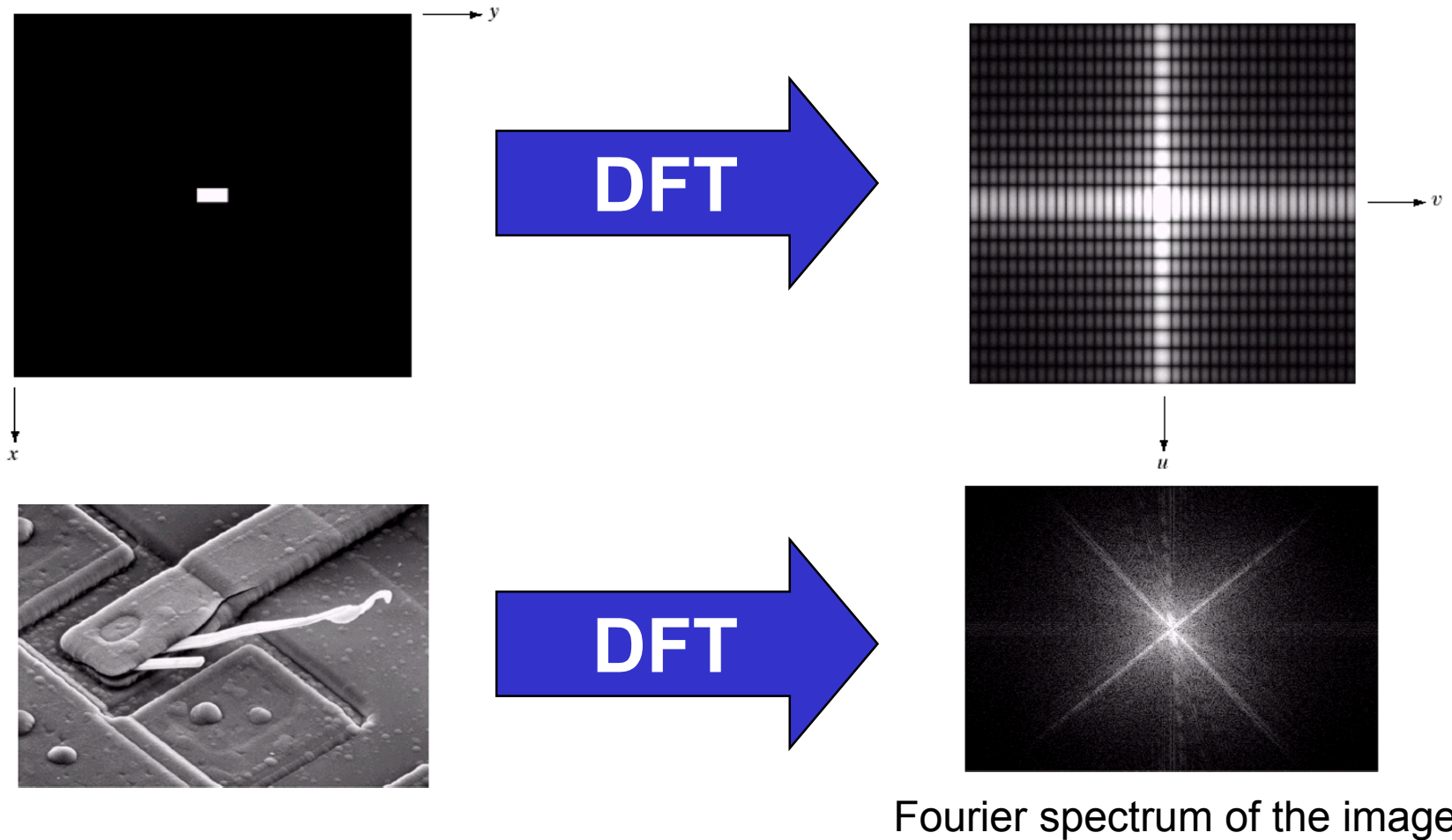
- Any function that periodically repeats itself can be expressed as a sum of sines and cosines of different frequencies each multiplied by a different coefficient
– a *Fourier series*

The *Discrete Fourier Transform* of $f(x, y)$,
for $x = 0, 1, 2 \dots M-1$ and $y = 0, 1, 2 \dots N-1$,
denoted by $F(u, v)$, is given by the equation:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(ux/M + vy/N)}$$

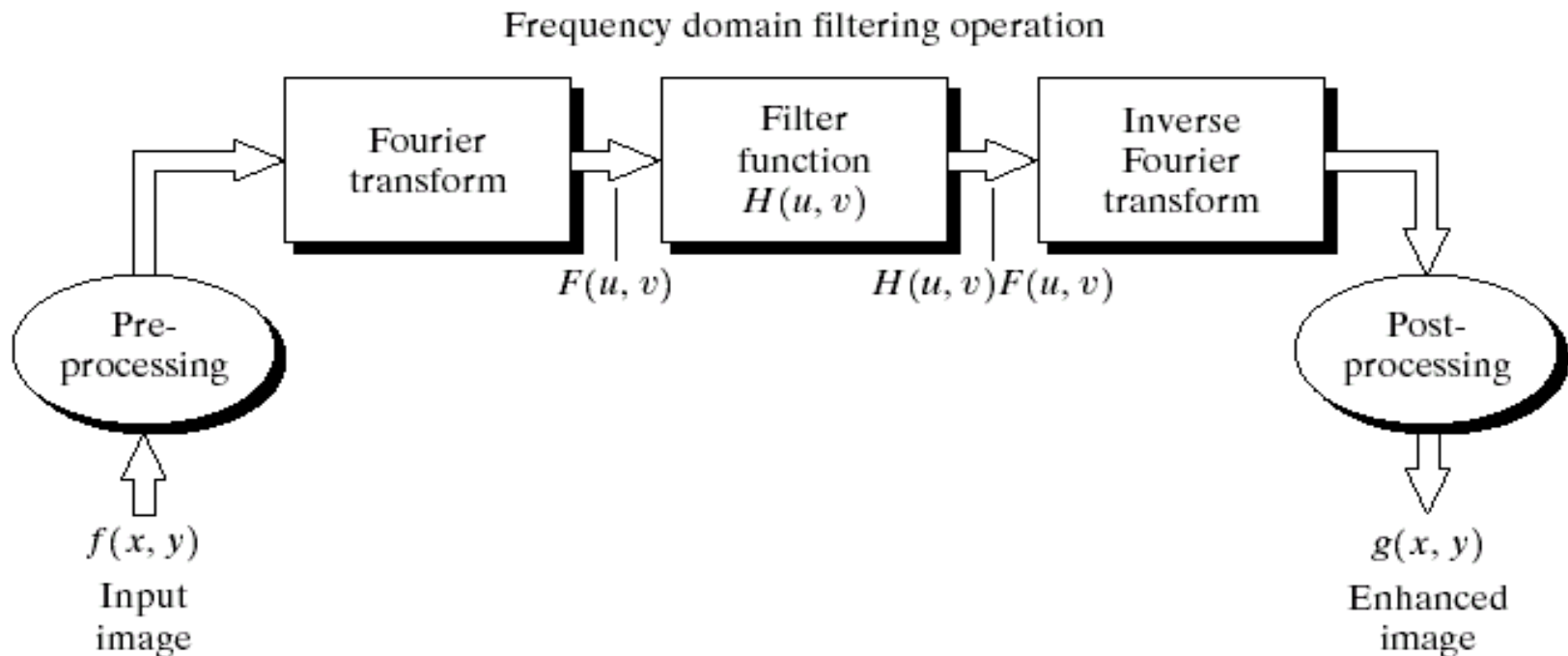
for $u = 0, 1, 2 \dots M-1$ and $v = 0, 1, 2 \dots N-1$.

- The DFT of a 2D image can be visualised by showing the spectrum of the images component frequencies

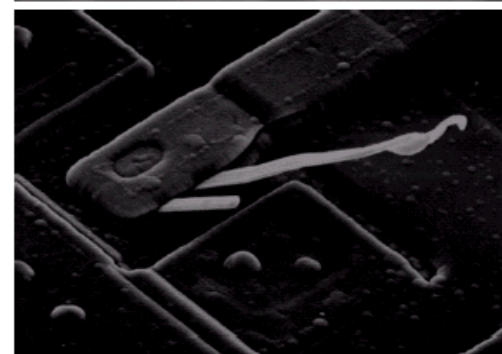
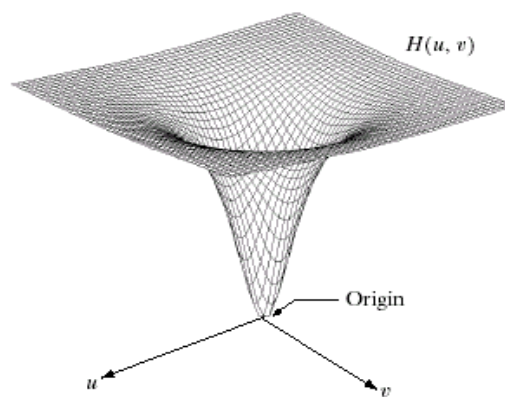
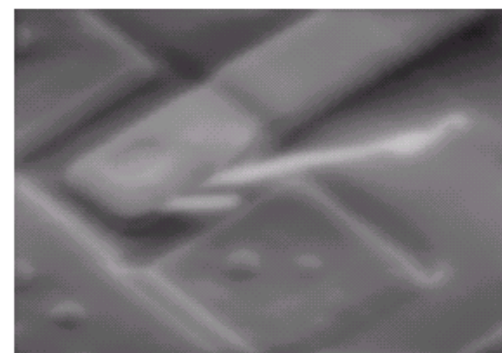
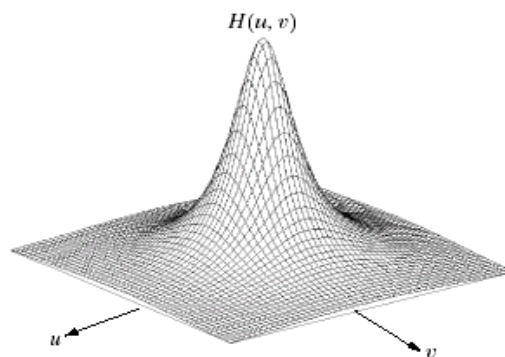
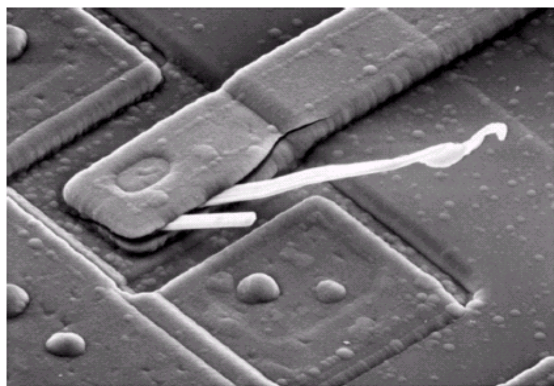


To filter an image in the frequency domain:

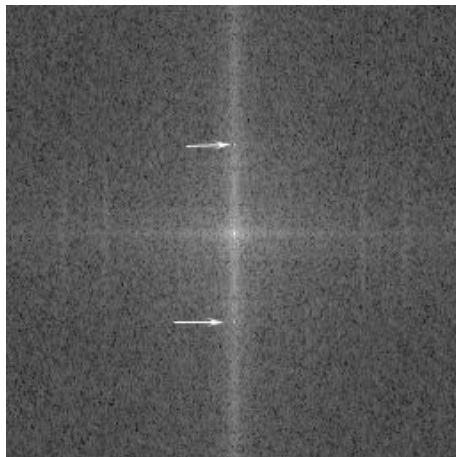
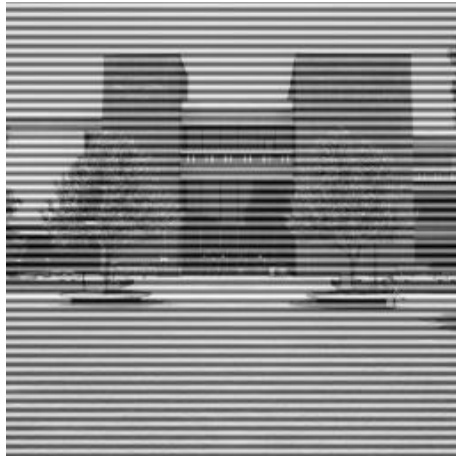
1. Compute $F(u,v)$ the DFT of the image
2. Multiply $F(u,v)$ by a filter function $H(u,v)$
3. Compute the inverse DFT of the result



Low Pass Filter



High Pass Filter



2D Fourier transform



After filtering